

# Pattern Recognition via Hopfield Network & Fuzzy Logic

A hybrid soft computing project implementing a Discrete Hopfield Network for auto-associative memory, combined with a Fuzzy Inference System to dynamically generate environmental noise. This project demonstrates content-addressable memory retrieval under varying conditions of uncertainty.

## 1. System Architecture

The project combines two fundamental Soft Computing paradigms:

- **Fuzzy Logic Pre-processor:** Evaluates an abstract "Environmental Interference" score (0-10) through triangular membership functions. It applies a defuzzified rule base to determine exactly what percentage of a target image's pixels should be corrupted.
- **Hopfield Network:** Employs a single-layer recurrent neural network. It memorizes

$5 \times 5$

- bipolar image vectors and uses an asynchronous update mechanism to reconstruct original patterns from the fuzzy-corrupted inputs.

## 2. Mathematical Model

### A. Hebbian Learning (Training Phase)

The Hopfield network stores a set of bipolar patterns ( $-1$  and  $1$ ). The weight matrix  $W$  is constructed using the Hebbian learning rule, ensuring that neurons do not connect to themselves.

$$W = \frac{1}{N} \sum_{k=1}^P (x_k \cdot x_k^T) - I$$

*(Where  $x$  is the stored pattern vector,  $P$  is the total patterns, and  $I$  is the identity matrix)*

## B. Network Recall & Energy Function

During recall, neurons asynchronously update their states based on the weighted sum of inputs from other neurons.

$$s_i(t + 1) = \text{sign}\left(\sum_{j \neq i} W_{ij} s_j(t)\right)$$

The network is guaranteed to converge to a local minimum because every state update strictly decreases the network's global energy (Lyapunov function):

$$E = -\frac{1}{2} \sum_i \sum_j W_{ij} s_i s_j$$

## 3. Project Relevance

This implementation goes beyond standard dataset classification by addressing Recurrent Neural Networks (RNNs) and dynamic system stability. The addition of Fuzzy Logic simulates real-world uncertainty (such as faulty factory sensors or noisy transmission channels), elevating the implementation to a robust, hybrid soft computing application suitable for postgraduate evaluation.

## 4. How to Run

```
# Install dependencies
pip install numpy matplotlib

# Execute the model
python3 fuzzy_hopfield_network.py
```

This will train the network, pass an image through the fuzzy noise injector, run the Hopfield retrieval, and generate `hopfield_fuzzy_result.png` showing the visual reconstruction matrix.